SOFT THINKING AT SMALL SCALE: DIAGNOSING AND REPAIRING ITS LIMITATIONS

Anarva Kamdar
M.Tech. (Artificial Intelligence), Indian Institute of Technology Kharagpur
Group Number: 33 Roll Number: 25AI60R24
Course: Foundations of Large Language Models (AI60213)

ABSTRACT

Soft Thinking is a training-free way to let a reasoning model "think between words". Instead of picking one token at every thinking step, the model feeds back a *concept token*: a probability-weighted mix of token embeddings. The authors report gains on 32B and 70B models, but they also name the method's main weakness themselves: the model has never seen such mixed inputs during training. We study the method on a small model (Qwen3-1.7B) and the GSM8K math benchmark, and ask three questions: does it still work, can we measure its weakness, and can we fix it? We re-implement Soft Thinking (89.2% accuracy) and introduce the *superposition gap*, which measures how far the model's reaction to a mixed input is from the mix of its reactions to each candidate word. The gap is clearly present: at 90% of mixed steps the model simply follows its most likely word, and mixed inputs are only $0.85\times$ the size of a real token. We then try four repairs. (1) A small *concept adapter* (0.26M parameters, no labels) reduces the gap significantly (-6.8%, $p < 10^{-16}$) while keeping accuracy unchanged. (2) A version trained on its own outputs does not reduce the gap further. (3) Re-tuning the paper's entropy-based stopping rule (Cold Stop) for a small model saves only 1.8% of tokens, even though about 60% of the thinking happens after the answer is already written. (4) An answer-aware early exit removes about half of the thinking (53%) with no significant loss in accuracy, across six independent sets of traces. Overall, closing the gap does not raise accuracy at this scale, but stopping at the right time makes small-scale Soft Thinking roughly twice as efficient.

1 INTRODUCTION

Reasoning models such as DeepSeek-R1 and Qwen3 solve problems by writing a long chain of thought before they answer (Wei et al., 2022). At every step of this chain the model computes a full probability distribution over its vocabulary, and then throws most of it away by committing to a single token. The second-best idea, the alternative phrasing, the "maybe it is 8, maybe it is 12" uncertainty: all of it is discarded before the next step.

Soft Thinking (Zhang et al., 2025) keeps that information. During thinking, it does not pick one token. It feeds the model the probability-weighted average of the embeddings of the likely tokens, which the authors call a *concept token*. The idea is simple, needs no training, and the authors report up to 2.48 points higher accuracy and up to 22.4% fewer tokens on 32B and 70B models.

The authors are also open about the method's main risk. A language model was only ever trained on real tokens, so a concept token is an input it has never seen, an *out-of-distribution* input. They add a stopping rule, *Cold Stop*, to limit the damage, and end their paper by suggesting "training-based approaches to adapt the concept token" as future work.

This project starts from that open question. We ask:

1. Does Soft Thinking still work on a **small** model that fits on a free GPU?
2. Can we **measure** its out-of-distribution problem instead of only describing it?
3. Can we **repair** the problems we find, and does the repair help?

Contributions.

- A faithful, batched re-implementation of Soft Thinking for Qwen3-1.7B on GSM8K, reaching 89.2% accuracy (Section 6).
- The **superposition gap**, a precise and cheap measure of how badly the model misreads a concept token, computed exactly at every layer with a "probe token" trick. We find that the small model is *single-threaded*: it reads 90% of mixed inputs as just their most likely word (Section 7).
- A **concept adapter**: a 0.26M-parameter module, trained without labels while the language model stays frozen, that significantly reduces the gap without changing accuracy (Section 8). A second, "calibrated" version trained on its own outputs serves as an ablation.
- Two **stopping-rule** studies. Re-tuning Cold Stop for a small model barely helps, because confidence does not tell us when the answer is ready. An **answer-aware early exit** removes about half of the thinking with no significant accuracy loss (Section 9).

All code is available in five Kaggle notebooks at `https://github.com/AnarvaKamdar1/FLLM_Project`.

2 Related work

Reasoning in continuous space. COCONUT (Hao et al., 2024) feeds the model's last hidden state back as its next input, but this needs extensive training. Soft Thinking (Zhang et al., 2025) avoids training by staying inside the convex hull of the real token embeddings. Two follow-up works study Soft Thinking further. Wu et al. (2026) show that large models mostly follow the top token of the mixture ("single-threaded reasoning") and add randomness to the mixture. SwiReasoning (Shi et al., 2026) switches between soft and normal thinking based on entropy. Both change *which* mixture is fed. Our adapter instead changes *how the model reads* the mixture.

Adapters. Small trainable modules added to a frozen network are a standard way to adapt large models cheaply (Houlsby et al., 2019). Our adapter is applied only to the input of mixed thinking steps.

Overthinking and early exit. Reasoning models often keep thinking long after they have found the answer (Chen et al., 2024). Early-exit methods stop the chain when the model's tentative answer becomes stable (Liu & Wang, 2025) or confident (Yang et al., 2026). These methods were developed for normal chain-of-thought; we apply answer-convergence to Soft Thinking, where the thinking is made of concept tokens.

3 Background: what Soft Thinking does

Let $E \in \mathbb{R}^{|V| \times d}$ be the input embedding matrix and $\mathbf{e}(k)$ the embedding of token k. In normal chain-of-thought, at thinking step t the model produces a distribution p_t over the vocabulary, picks one token x_{t+1} from it, and feeds back $\mathbf{e}(x_{t+1})$.

Concept tokens. Soft Thinking keeps the distribution itself. It first filters p_t (top-k, top-p and min-p) and renormalises it, giving the concept token $\mathbf{c}_t$. It then feeds back the weighted average of embeddings:

$$\tilde{\mathbf{e}}_t \;=\; \sum_k \mathbf{c}_t[k]\, \mathbf{e}(k) \;=\; \mathbf{c}_t^\top E. \tag{1}$$

If $\mathbf{c}_t$ puts all its weight on one token, Equation 1 gives back ordinary chain-of-thought. Only the thinking phase is soft: once thinking ends, the answer is written with normal tokens. The authors' intuition is that a concept token carries several possible next steps at once, so the model can follow several reasoning paths in parallel.

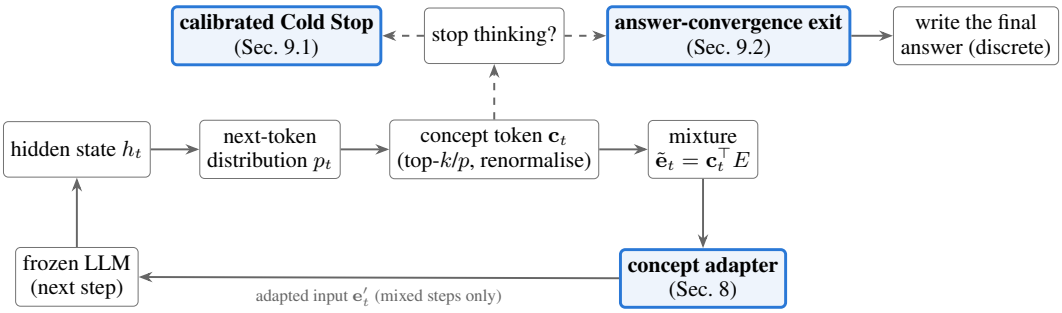


Figure 1: Soft Thinking (grey boxes) and where our interventions act (blue). The concept adapter recalibrates the mixed input before the frozen model reads it. The two stopping rules decide when thinking ends: the paper's entropy-based Cold Stop, re-tuned for a small model, and an answer-aware early exit.

Cold Stop. Because concept tokens are unfamiliar inputs, long soft chains can become unstable, for example by repeating themselves. Cold Stop measures the entropy $H(\mathbf{c}_t) = -\sum_k \mathbf{c}_t[k] \log \mathbf{c}_t[k]$ at every step. If it stays below a threshold τ for k steps in a row, thinking ends. The paper tunes τ and k for 32B and 70B models, with k between 128 and 1024 steps.

Figure 1 shows the full pipeline and where our four interventions plug in.

4 The limitations we study

L1: Concept tokens are out of distribution (stated by the authors). The model never saw a mixed embedding during training, so there is no guarantee it reads $\tilde{\mathbf{e}}_t$ as the intended blend of candidates. The paper describes this problem but never measures it.

L2: Cold Stop is tuned for large models. Its thresholds were chosen for 32B and 70B models. It is not clear that they work for a 1.7B model.

L3: The efficiency claim is untested at small scale. The paper reports up to 22.4% fewer tokens. For a small model, it is unknown how much of the thinking is actually needed.

We address L1 with a measurement (the superposition gap) and a repair (the concept adapter), and L2–L3 with two stopping rules.

5 Experimental setup

Model and data. We use `Qwen/Qwen3-1.7B` (Qwen Team, 2025) in thinking mode, frozen and in fp16, on one NVIDIA T4 GPU (Kaggle). We evaluate on the first 250 questions of the GSM8K test set (Cobbe et al., 2021). A fixed random sample of 400 GSM8K *training* questions is used only to collect thinking traces for training and calibration; their answers are never used to train anything.

Soft Thinking settings. These are identical for every system: temperature 0.6, top-k 15, top-p 0.95, min-p 0.001, Cold Stop $\tau = 0.01$ and $k = 128$ (unless stated otherwise), at most 2048 thinking tokens and 512 answer tokens. The final answer is written greedily and read from `\boxed{}`.

Implementation. The official code patches the SGLang server. We wrote our own batched version in PyTorch and Hugging Face Transformers, decoding 16 questions at a time with left padding and explicit position ids. Decoding is deterministic, so a question always produces the same thinking trace. We use this to replay saved traces exactly (Sections 9.1 and 9.2).

Statistics. Every system answers the same questions, so all comparisons are *paired*. For accuracy we use an exact McNemar test (McNemar, 1947) on the questions where exactly one system is

correct. For token counts and gap measurements we use a Wilcoxon signed-rank test (Wilcoxon, 1945) on per-question values. We call a difference significant when $p < 0.05$.

6 Reproducing Soft Thinking

Soft Thinking works well at small scale. On the 250 test questions it reaches **89.2%** accuracy with 1,342 thinking tokens on average (Table 3). 72% of questions end thinking on their own with `</think>`, and 28% reach the 2,048-token budget. With the paper's settings, **Cold Stop never fires**. Only **21%** of thinking steps are real mixtures (at least two candidate tokens survive filtering); the other 79% behave like ordinary tokens.

7 Measuring the out-of-distribution problem: the superposition gap

Definition. Soft Thinking quietly assumes that *mixing embeddings is the same as mixing behaviours*. In other words, given the same prefix, feeding the mixture $\tilde{\mathbf{e}}_t$ should make the model behave like the weighted mix of what it would do for each candidate token. We test this directly. Let $h_\ell(\cdot)$ be the hidden state at layer ℓ for a given input, with everything before step t held fixed. The *superposition gap* at layer ℓ is

$$\mathrm{gap}_\ell \;=\; \frac{\left\| h_\ell(\tilde{\mathbf{e}}_t) - \sum_k \mathbf{c}_t[k]\, h_\ell(\mathbf{e}(k)) \right\|}{\left\| \sum_k \mathbf{c}_t[k]\, h_\ell(\mathbf{e}(k)) \right\|}. \tag{2}$$

We also report the *output gap*, the KL divergence between the intended mix of next-token distributions $\sum_k \mathbf{c}_t[k]\, p(\cdot \mid \mathbf{e}(k))$ and what the model actually predicts. Two further checks round this out: the *norm ratio* $\|\tilde{\mathbf{e}}_t\| / \sum_k \mathbf{c}_t[k] \|\mathbf{e}(k)\|$, and whether $h(\tilde{\mathbf{e}}_t)$ is closer to the superposition or to the top candidate alone.

Computing it cheaply. Equation 2 needs the model's response to every candidate token given the exact prefix of step t. Recomputing each prefix would take hundreds of forward passes per question. Instead, we run one forward pass over the real trace to fill the key–value cache. We then append all "probe tokens" $\mathbf{e}(k)$ at once, each with the position id of its step and an attention mask that lets it see only the tokens before that step. A sanity check against brute-force recomputation gives a relative error below 3.5×10^{-3} at every layer, which is fp16 noise.

Findings. The gap is real and large (Figure 2, Table 1):

- **Mixtures are too short.** On average they are only $0.85\times$ the size of a real token, falling to about $0.5\times$ for uncertain steps.
- **The model is single-threaded.** At **90%** of mixed steps, its final hidden state is closer to its reaction to the top-1 candidate than to the intended superposition (mean distance 0.079 vs. 0.206). This repeats, at 1.7B scale, what Wu et al. (2026) found for large models.
- **The worst cases mix incompatible words.** The largest output gaps come from mixtures such as "`five`" (0.56) with a bare space (0.44): two continuations that cannot be meaningfully averaged.
- **The gap does not predict failure.** Wrong answers have a slightly *lower* median output gap (3.7 vs. 4.2), so a larger gap does not by itself mean a wrong answer.

8 Repair 1: the concept adapter

Idea. If the model misreads a mixture, we can change the mixture slightly before the model reads it, so that the model's response moves toward the intended superposition. We do this with a small trainable module while the language model stays completely frozen.

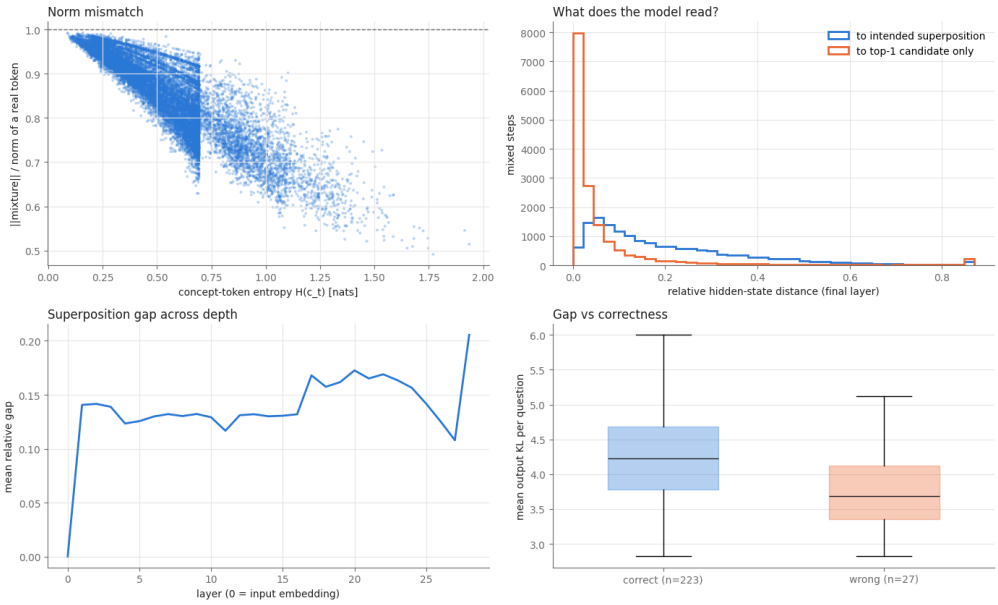


Figure 2: The superposition gap of Soft Thinking on Qwen3-1.7B (250 GSM8K test questions, 15,909 mixed steps). **Top left:** mixtures are shorter than real tokens, and more so when the concept token is more uncertain. **Top right:** the model's final hidden state is much closer to its reaction to the top-1 candidate (orange) than to the intended superposition (blue). **Bottom left:** the gap exists at every layer and is largest at the output. **Bottom right:** wrongly answered questions do *not* have a larger gap.

Architecture. For every mixed concept token (at least two candidates), with $s_t = \mathrm{RMS}(\tilde{\mathbf{e}}_t)$,

$$\mathbf{y}_t = \tilde{\mathbf{e}}_t + s_t \cdot W_{\text{up}}\,\mathrm{GELU}\big(W_{\text{down}}\,\tilde{\mathbf{e}}_t / s_t\big), \tag{3}$$

$$\mathbf{e}'_t = \mathbf{y}_t \cdot \min\Big(1,\ \frac{\sum_k \mathbf{c}_t[k]\,\|\mathbf{e}(k)\|}{\|\mathbf{y}_t\|}\Big). \tag{4}$$

W_{down} and W_{up} have rank 64, about 0.26M parameters in total. W_{up} starts at zero, and by the triangle inequality the norm cap is inactive for the plain mixture. So **before training the adapter is exactly the original Soft Thinking**, which we verified in code. The cap stops the adapter from pushing inputs beyond the size of a real token.

Training objective. The adapter is trained to close the superposition gap at every layer:

$$\mathcal{L} = \frac{1}{L+1}\sum_{\ell=0}^{L} \frac{\big\|h_\ell(\mathbf{e}'_t) - \sum_k \mathbf{c}_t[k]\,h_\ell(\mathbf{e}(k))\big\|^2}{\big\|\sum_k \mathbf{c}_t[k]\,h_\ell(\mathbf{e}(k))\big\|^2}. \tag{5}$$

Three design choices matter here.

- **All layers.** Every input also writes keys and values at every layer, which all later tokens read. A loss on the next-token prediction alone would leave these unconstrained. In early experiments such a loss made the adapter inflate its inputs, and generation collapsed into loops.
- **No hedging.** At the last layer, the target is a mix of *logits*, which keeps predictions sharp instead of rewarding a spread-out guess.
- **Built-in trust region.** Layer $\ell = 0$ is the input itself, so the first term keeps $\mathbf{e}'_t$ close to $\tilde{\mathbf{e}}_t$ without any extra hyperparameter.

The targets come from the frozen model's own responses to each candidate, so **no labels are needed**. We collect Soft Thinking traces on the 400 training questions (360 for training, 40 for validation)

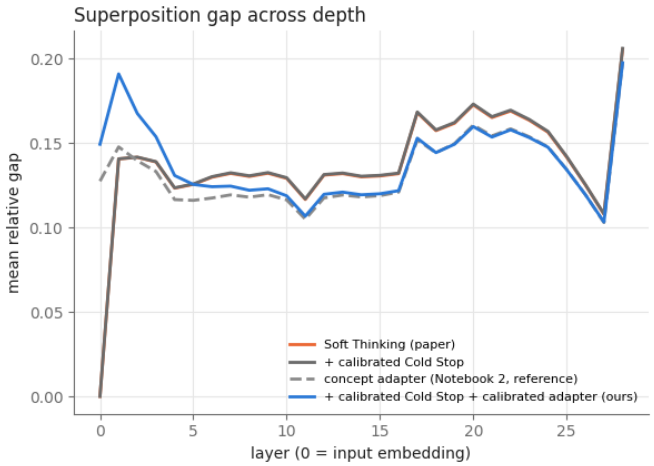


Figure 3: Mean superposition gap at each layer during real generation. Both adapters (dashed grey: concept adapter; blue: calibrated adapter) lower the gap in the middle layers. Neither moves it much at the final layer. The paper's method (orange) is hidden under the calibrated-Cold-Stop line (dark grey), because the two are nearly identical.

and train for 3 epochs with AdamW (learning rate 10^{-3}, cosine schedule, gradient clipping 1.0). Each trace needs one cached forward pass, one probe pass and one small backward pass over up to 64 mixed steps. Training takes 18 minutes on a T4.

Results. Table 1 compares the gap *during real generation* on the 250 test questions, and Figure 3 shows it layer by layer.

- **The gap shrinks significantly.** Averaged over layers it falls from 0.1425 to 0.1328 (-6.8%, $p = 3.6 \times 10^{-17}$, lower on 74% of questions). The output KL falls by 4.3% ($p = 3 \times 10^{-4}$).
- **Most of the gain is in the middle layers**, whose keys and values later tokens read. Much of it fades at the final layer (-3.7%). The price is a larger gap at the input layer, where the adapter moves the mixture by about 13%.
- **Accuracy and behaviour are unchanged** (Table 3): 87.6% vs. 89.2%, with 11 questions lost and 7 gained ($p = 0.48$). Thinking length (1,359 vs. 1,342 tokens), stop reasons, repetition and decoding time (80.8 vs. 82.6 min) stay the same.

Calibrated adapter (ablation). Two weaknesses of the first adapter are that the gain fades at the final layer, and that it is trained on traces generated *without* the adapter. We therefore trained a second, "calibrated" version:

- it is trained on traces generated *with* the first adapter, i.e. on-policy, in the spirit of DAgger (Ross et al., 2011);
- it starts from the first adapter's weights;
- its layer weights in Equation 5 rise with depth, from 0 for the first layers to 2 for the last.

It also reduces the gap significantly against the same stopping rule (all layers -3.4%, final layer -4.1%, output KL -5.0%, all $p \leq 10^{-4}$). Its accuracy is the highest we observed (90.4%), but the difference is not significant (12 gained, 9 lost, $p = 0.66$). However, it does **not** beat the first adapter at the final layer (0.1975 vs. 0.1978, $p = 0.65$), and the depth weighting makes the early layers worse (Figure 3). The final-layer gap seems to have a floor near 0.197 that an input-side adapter cannot move.

Table 1: Superposition gap measured during real generation on the same 250 test questions. Arrows show the better direction; "top-1 wins" is the share of mixed steps where the model's state is closer to the top candidate than to the superposition. p-values compare each adapter with the system it is built on (Wilcoxon, per-question means).

System	Norm ratio	Gap, all layers ↓	Gap, final ↓	Output KL ↓	Top-1 wins	Argmax agree ↑
Soft Thinking (paper)	0.852	0.1425	0.2055	4.20	90.1%	92.7%
+ concept adapter	0.847	**0.1328**	0.1978	4.01	89.5%	**93.1%**
p vs. paper		3.6×10^{-17}		3×10^{-4}		
Calibrated Cold Stop	0.851	0.1428	0.2060	4.21	90.1%	92.8%
+ calibrated adapter	0.847	0.1380	**0.1975**	**3.99**	**87.1%**	92.6%
p vs. calibrated Cold Stop		2.8×10^{-5}	1.4×10^{-4}	7.6×10^{-5}		

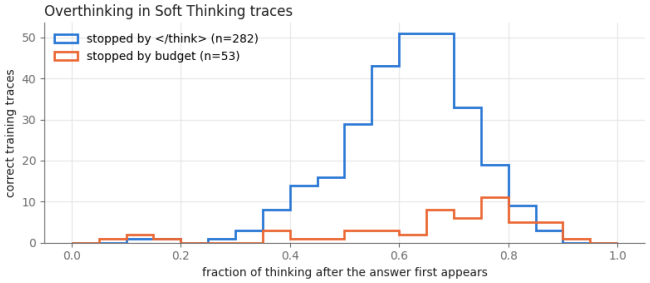


Figure 4: Share of each correct training trace that comes *after* the answer first appears. Most traces spend more than half of their thinking re-checking an answer they already have.

9 Repairs 2 and 3: When should thinking stop?

The model overthinks. For every correct training trace, we find the first thinking step at which the correct answer has been written in the readable trace (the most likely token at each step). On average, **62% of the thinking happens after the answer is already there** (Figure 4). This holds for traces that stop on their own (61%) and for traces that hit the budget (65%). So there is plenty of room to save tokens, if a rule can tell when the answer is ready.

9.1 Calibrated Cold Stop

Method. Because decoding is deterministic, we can work out exactly where Cold Stop *would* have fired on a saved trace for any (τ, k). Up to that point the trace is identical. On the 400 training traces we try 40 settings ($\tau \in \{0.01, 0.05, 0.1, 0.25, 0.5\}$, $k \in \{16, \ldots, 256\}$) and record two numbers for each:

- the share of thinking tokens it would save;
- the share of correct traces it would cut before the answer appears ("harm").

The selection rule was fixed in advance: take the largest saving with harm at most 2%. The test set is never used to choose.

Results. Savings and harm grow almost one-for-one (Figure 5). For example, $k = 64$ saves 8% of tokens but cuts 5% of correct traces too early, and $k = 48$ saves 25% but harms 19%. The selected setting ($\tau = 0.25$, $k = 96$) gives on the test set:

- identical answers on all 250 questions (accuracy 89.2%);
- only 1.8% fewer thinking tokens. This is statistically significant ($p = 0.018$) but too small to matter.

Our replay predicted the exact cut point for 100% of questions, which confirms the deterministic-replay method. The lesson is that **confidence is the wrong signal**: the model is confident in many places during reasoning, not only after it has found the answer.

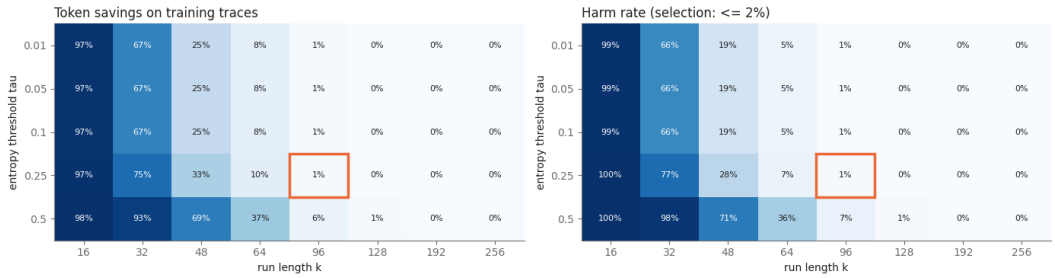


Figure 5: Cold Stop on the 400 training traces: token savings (left) and harm, i.e. correct traces cut before the answer appears (right), for each setting. Savings and harm rise together, so within 2% harm only the outlined setting ($\tau = 0.25$, $k = 96$) qualifies, saving about 1%.

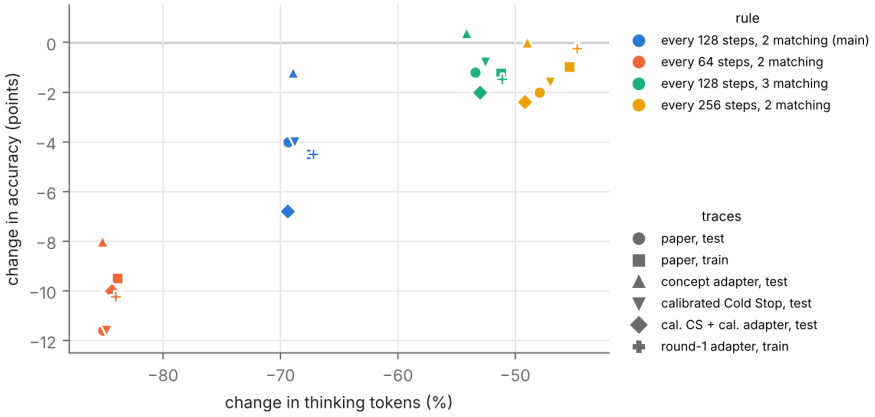


Figure 6: Answer-convergence exit: change in thinking tokens against change in accuracy for four rules (colour) on six sets of saved traces (shape). Asking for three matching answers (green) or probing every 256 steps (yellow) removes about half of the thinking for a loss of 0–2 points; none of these losses is significant.

9.2 Answer-convergence early exit

Method. Instead of asking "is the model confident?", we ask "has the answer stopped changing?", following Liu & Wang (2025). Every 128 thinking steps:

1. We briefly ask the model for its current answer by appending `</think>\n\nThe final answer is \boxed{` and reading up to 12 greedy tokens.
2. We then discard these probe tokens.
3. When the last two probes give the same number, thinking stops and the model writes its final answer exactly as before.

The method needs no training and no labels. Using the same deterministic replay and parallel probe trick, we evaluate it on *every* set of saved traces: about 1,650 traces from six systems. Every final answer is really generated by the model from the exit point, so no proxy is involved. Probe tokens are counted as extra cost.

Results. Table 2 and Figure 6 summarise the results.

- **The rule fixed in advance (every 128 steps, 2 matching) is too eager.** It removes 69% of the thinking ($p \approx 10^{-42}$) but costs 4 points of accuracy (89.2% → 85.2%, $p = 0.031$). Inspecting the lost questions shows the cause: early probes often return placeholder numbers such as 0, 1 or 5, and two of them can agree by chance.

Table 2: Answer-convergence exit on Soft Thinking traces. Token change excludes probe overhead (about +3 points when included). Accuracy p: McNemar against the same traces without the exit. Every token change has $p < 10^{-30}$.

Traces	Exit rule	Accuracy	Acc. p	Thinking tokens	Token change
Paper, test (250)	none	89.2%	–	1,341	–
	every 128, 2 matching (fixed in advance)	85.2%	0.031	411	−69.4%
	every 128, 3 matching (chosen on train)	**88.0%**	0.45	625	**−53.4%**
	every 256, 2 matching	87.2%	0.18	698	−47.9%
Paper, train (400)	none	89.3%	–	1,288	–
	every 128, 3 matching	88.0%	0.40	629	−51.2%
Concept adapter, test	every 128, 3 matching	87.6% → 88.0%	1.00	1,358 → 622	−54.2%
Calibrated Cold Stop, test	every 128, 3 matching	89.2% → 88.4%	0.69	1,316 → 625	−52.6%
Cal. Cold Stop + cal. adapter, test	every 128, 3 matching	90.4% → 88.4%	0.27	1,338 → 629	−53.0%

Table 3: All systems on the same 250 GSM8K test questions (Qwen3-1.7B, one T4). Accuracy p: McNemar against the paper's method. "Budget" is the share of questions that reach the 2,048-token limit. Exit = answer-convergence exit, every 128 steps, 3 matching.

System	Trainable params	Accuracy	Acc. p	Thinking tokens	Budget
Soft Thinking (paper)	0	89.2%	–	1,342	28.0%
+ concept adapter (ours)	0.26M	87.6%	0.48	1,359	–
+ calibrated Cold Stop (ours)	0	89.2%	1.00	1,317	26.8%
+ calibrated Cold Stop + calibrated adapter (ours)	0.26M	**90.4%**	0.66	1,339	27.2%
+ answer-convergence exit (ours)	0	88.0%	0.45	**625**	–

- **A stricter rule fixes this.** Among the other rules, we choose using the *training* questions only: the rule with the largest saving and no significant accuracy loss is "every 128 steps, 3 matching" (-51% tokens, $p = 0.40$). On the test questions it removes **53%** of the thinking with no significant accuracy change (89.2% → 88.0%, $p = 0.45$).
- **The stricter rule holds everywhere.** The same holds for all six sets of traces, with accuracy changes between -2.0 and $+0.4$ points and every $p \geq 0.27$.
- **Probes are cheap.** They add about 40 tokens per question, against more than 700 removed.

We stress that the main rule was set before the run, and that the stricter rule was then picked on training questions, never on the test set.

10 Overall comparison

Table 3 puts all systems side by side on the 250 test questions. Accuracy stays within about ± 1.5 points of the paper for every training-based change, and none of these differences is significant. Efficiency changes only when the stopping rule knows about the answer.

11 Discussion and limitations

What worked.

- **The diagnosis.** The superposition gap turns the paper's vague "out-of-distribution" worry into a number we can measure at every layer. It shows a clear and repeatable picture: the small model is single-threaded, and mixtures are too short.
- **The concept adapter** closes part of this gap with very little cost, and two independent training runs both reduced it significantly.
- **The answer-convergence exit** delivers the efficiency that the paper promised but that Cold Stop cannot provide at small scale: about half the thinking for the same accuracy.

What did not work, and why.

- **Closing the gap did not raise accuracy.** At this scale, how precisely the model reads a mixture is evidently not what limits its accuracy. Since the model follows the top candidate 90% of the time anyway, Soft Thinking behaves much like greedy reasoning here.
- **The final-layer gap barely moved.** Neither on-policy training nor depth weighting changed it, which suggests a floor that an input-side adapter cannot reach.
- **Cold Stop cannot be rescued by tuning**, because entropy measures confidence, not completion.

Limitations.

- **Scope.** We use one model, one dataset and one run per system. GPU training is not bit-exact, and our two adapter runs differed by about a point in accuracy.
- **Statistical power.** With 250 test questions, only accuracy differences of roughly 5 points could be detected.
- **Off-policy training.** The adapter is trained on traces it did not generate.
- **Exact replay.** The early-exit results rely on deterministic decoding, which we verified at 100%.
- **Numeric answers.** The answer probe is designed for numeric answers.
- **Novelty of the exit.** The answer-convergence idea comes from prior work on normal chain-of-thought; our contribution is applying and evaluating it for Soft Thinking.

Future work. Several directions follow naturally:

- adapters that act inside the model, not only on its input, to reach the final layers;
- a minimum-thinking rule before the first answer probe, to avoid early false agreement;
- other reasoning models and harder benchmarks, where the gap may matter more.

12 Conclusion

We tested Soft Thinking on a small reasoning model and looked closely at the weakness its authors pointed out. The method still works at 1.7B parameters, but the model mostly ignores the "soft" part: it reads a mixture as its most likely word, and spends most of its thinking re-checking answers it already has. A tiny, label-free concept adapter measurably closes the gap between what the model does and what Soft Thinking intends, without hurting accuracy. Closing this gap alone does not make the model more accurate. Stopping at the right time is what helps: an answer-aware exit halves the thinking at no significant cost, making small-scale Soft Thinking about twice as efficient.

Reproducibility statement

All experiments run on a single free Kaggle T4 GPU. The five notebooks, their saved outputs and the exact configuration of every run are available at `https://github.com/AnarvaKamdar1/FLLM_Project`. Notebook 1 reproduces Soft Thinking and the gap diagnosis; Notebook 2 trains the concept adapter; Notebook 3 calibrates Cold Stop; Notebook 4 trains the calibrated adapter; Notebook 5 evaluates the answer-convergence exit. Each notebook has a short "QUICK" mode for testing and a "FULL" mode for the reported numbers.

References

Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do NOT think that much for 2+3=? On the overthinking of o1-like LLMs. *arXiv preprint arXiv:2412.21187*, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John

Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning (ICML)*, 2019.

Xin Liu and Lu Wang. Answer convergence as a signal for early stopping in reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 17896–17907, 2025.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

Dachuan Shi, Abedelkadir Asi, Keying Li, Xiangchi Yuan, Leyan Pan, Wenke Lee, and Wen Xiao. SwiReasoning: Switch-thinking in latent and explicit for pareto-superior reasoning LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.05069.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

Junhong Wu, Jinliang Lu, Zixuan Ren, Gangqiang Hu, Zhi Wu, Dai Dai, and Hua Wu. LLMs are single-threaded reasoners: Demystifying the working mechanism of soft thinking. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2508.03440.

Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2504.15895.

Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, Shuohang Wang, Yelong Shen, and Xin Eric Wang. Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.15778.

A Implementation details

Probe tokens. To measure $h_\ell(\mathbf{e}(k))$ given the prefix of step t, we first run the model once over the real trace and keep its key–value cache. We then append one probe token per (step, candidate) pair. Each probe gets the position id of its step, and a boolean attention mask lets it see only cached positions before its step plus itself. This gives all candidate responses for a trace in a single extra forward pass. The same trick, with block-causal masks over several tokens, runs all answer probes of a trace in parallel for the early exit.

Runtime. On one T4, Soft Thinking on 250 test questions takes about 80 minutes. Collecting 400 training traces takes about 2 hours, and adapter training about 18 minutes. The full early-exit replay over six sets of traces took about 7 hours, mostly to generate final answers at each exit point.

B All early-exit rules

Table 4: Settings shared by all experiments.

Setting	Value
Model	`Qwen/Qwen3-1.7B`, thinking mode, frozen, fp16
Hardware	$1 \times$ NVIDIA T4 (16 GB), Kaggle
Test set	GSM8K test, first 250 questions
Training traces	400 GSM8K training questions (fixed random sample, seed 0)
Concept token filter	temperature 0.6, top-k 15, top-p 0.95, min-p 0.001
Cold Stop (paper / calibrated)	$\tau = 0.01$, $k = 128$ / $\tau = 0.25$, $k = 96$
Thinking / answer budget	2,048 / 512 tokens; greedy answer from `\boxed{}`
Batch size	16 questions, left padding, explicit position ids
Adapter	rank 64, residual MLP with norm cap, 264,256 parameters
Adapter training	AdamW, lr 10^{-3}, cosine schedule, 3 epochs, gradient clip 1.0, 360/40 split
Mixed steps per trace	up to 64 (diagnosis and training)
Answer probe	every 64 steps computed; rules use every 64/128/256; 12 greedy tokens

Table 5: Every exit rule on every set of saved traces: accuracy without $\rightarrow$ with the exit (McNemar p), and the change in thinking tokens.

Traces	every 128, 2 match	every 64, 2 match	every 128, 3 match	every 256, 2 match
Paper, test	89.2→85.2 (0.031), −69%	89.2→77.6 (<0.001), −85%	89.2→88.0 (0.45), −53%	89.2→87.2 (0.18), −48%
Paper, train	89.3→84.8 (0.001), −67%	89.3→79.8 (<0.001), −84%	89.3→88.0 (0.40), −51%	89.3→88.3 (0.54), −45%
Concept adapter, test	87.6→86.4 (0.66), −69%	87.6→79.6 (<0.001), −85%	87.6→88.0 (1.00), −54%	87.6→87.6 (1.00), −49%
Calibrated Cold Stop, test	89.2→85.2 (0.031), −69%	89.2→77.6 (<0.001), −85%	89.2→88.4 (0.69), −53%	89.2→87.6 (0.29), −47%
Cal. CS + cal. adapter, test	90.4→83.6 (<0.001), −69%	90.4→80.4 (<0.001), −84%	90.4→88.4 (0.27), −53%	90.4→88.0 (0.15), −49%
Round-1 adapter, train	89.5→85.0 (0.005), −67%	89.5→79.3 (<0.001), −84%	89.5→88.0 (0.31), −51%	89.5→89.3 (1.00), −45%